Date: 27-10-2026

CO-2 - AT-3

ERROR ANALYSIS

COURES CODE: DSA0308

COUSRE NAME: NLP

Reg no: 192472313 **Name: Pavan Jana**

Question 1 – Morphological Analysis Error in Biomedical Dataset

1. Preprocessing the Dataset

The biomedical text dataset is preprocessed by converting text to lowercase, removing punctuation, and tokenizing sentences into individual words.

Example input:

Infections are caused by infectious microorganisms.

Pre-processed output:

infections, are, caused, by, infectious, microorganisms

2. Apply Porter Stemmer

```
from nltk.stem import PorterStemmer

ps = PorterStemmer()

words = ["infection", "infectious", "infected", "infect", "infections"]

for word in words:

    print(word, "->", ps.stem(word))
```

Output:

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [
AMD64)] on win32
Enter "help" below or click "Help" above for more information
>>>
================ RESTART: C:/Users/janap/OneDrive/Documents/nl
infection -> infect
infectious -> infecti
infected -> infect
infect -> infect
infections -> infect
>>>
```

3. Identify Incorrect Stemming

Original word	Stem	Error analysis
infection	infect	Derivational information is lost
infectious	infecti	Produces an unnatural stem
infected	infect	Inflectional suffix is removed
infect	infect	Correct base form
infections	infect	Plural information is removed

The Porter Stemmer reduces words using suffix-removal rules. However, it may remove meaningful parts of words or produce stems that are not valid dictionary words.

4. Inflectional and Derivational Morphology

- Inflectional morphology: Changes grammatical information without creating a fundamentally new word. For example, infected and infections contain grammatical forms.
- Derivational morphology: Creates a related word with a different meaning or grammatical category. For example, infect → infection and infectious.

Inflectional morphology can often be normalized safely for search, but derivational morphology may carry important meaning in biomedical reports. Therefore, lemmatization and morphology-aware indexing can improve retrieval accuracy.

Question 2 – Error Analysis of a Finite-State Morphological Parser in Social Media Data

1. Identify Words Producing Incorrect Analyses

The following words may be incorrectly analyzed by a simple finite-state morphological parser.

Word	Correct morphological analysis
happiest	happy + est
unbelievable	un + believe + able
running	run + ing
reordering	re + order + ing
smartphones	smart + phone + s
unreadable	un + read + able

A parser that recognizes only one prefix or suffix may fail to analyze words containing multiple affixes.

2. Why Does the Parser Fail?

The parser may fail because:

1. It recognizes only one affix at a time.
2. It does not support combinations of prefixes and suffixes.
3. It uses a limited list of roots and affixes.
4. It does not handle spelling changes, such as happy becoming happiest.
5. It may incorrectly split words into invalid roots.

3. Modified Finite-State Parser

The following Python program handles several prefixes and suffixes and includes a special rule for -iest.

```
prefixes = ["un", "re"]
```

```
suffixes = ["est", "ing", "able", "s"]
def parser(word):
    root = word
    found_prefix = ""
    found_suffix = ""
    for prefix in prefixes:
        if root.startswith(prefix) and len(root) > len(prefix):
            found_prefix = prefix
            root = root[len(prefix):]
            break
    if root.endswith("iest") and len(root) > 4:
        root = root[:-4] + "y"
        found_suffix = "iest"
    else:
        for suffix in suffixes:
            if root.endswith(suffix) and len(root) > len(suffix):
                found_suffix = suffix
                root = root[:-len(suffix)]
                break
    return found_prefix, root, found_suffix
words = [
    "happiest", "unbelievable", "running",
    "reordering", "smartphones", "unreadable"
]
```

for word in words:

print(word, "->", parser(word))4. Evaluate Parser Accuracy

Output:

```
Enter "help" below or click "Help" above for more information.
>>>
============== RESTART: C:/Users/janap/OneDrive/Documents/nl1.py ==============
happiest -> ('', 'happy', 'iest')
unbelievable -> ('un', 'believ', 'able')
running -> ('', 'runn', 'ing')
reordering -> ('re', 'order', 'ing')
smartphones -> ('', 'smartphone', 's')
unreadable -> ('un', 'read', 'able')
>>>
```

4. Evaluate Parser Accuracy

Parser accuracy can be measured by comparing its output with manually verified morphological analyses.

$$\text{Accuracy} = \frac{\text{Correct analyses}}{\text{Total words}} \times 100$$

The original parser's accuracy should be measured before modification, and the corrected parser's accuracy should be measured using the same test words and gold-standard analyses. Exact accuracy values require executing both versions against an annotated dataset.

5. Computational Complexity

Let \(n\) be the word length, \(p\) the number of prefixes, and \(s\) the number of suffixes.

The simple implementation checks a fixed number of prefixes and suffixes, so its time complexity is \(O(n)\) for a fixed rule set. If the number of rules grows, the cost can increase with the number of rules. A finite-state transducer with a fixed transition network can process a word in time proportional to its length.

Conclusion: Adding transitions for prefix-suffix combinations and spelling changes improves morphological analysis. However, a real parser should also validate roots against a lexicon.

Question 6 – Error Analysis of a Finite-State Morphological Parser

1. Identify the Logical Errors

Given code:

```
def parser(word):
    if word.endswith("s"):
        return word[:-2], "Plural Noun"
    else:
        return word, "Singular"
```

The errors are:

- word[:-2] removes two characters instead of one for regular plurals.
- Words ending in -es need special handling.
- Words ending in -ies usually change ies to y.
- Irregular plurals such as children cannot be handled by suffix rules alone.
- Words ending in s are not always plural nouns.

2. Corrected Parser

```
def parser(word):
    irregular = {
        "children": "child",
        "men": "man",
        "women": "woman",
        "teeth": "tooth",
        "feet": "foot",
        "mice": "mouse",
        "geese": "goose"
```

```
    }
    if word in irregular:
        return irregular[word], "Plural Noun"
    if word.endswith("ies") and len(word) > 3:
        return word[:-3] + "y", "Plural Noun"
    if word.endswith(("ches", "shes", "xes", "zes", "sses")):
        return word[:-2], "Plural Noun"
    if word.endswith("s") and not word.endswith("ss"):
        return word[:-1], "Plural Noun"
    return word, "Singular"
words = ["cars", "boxes", "cities", "children"]
for word in words:
    print(word, "->", parser(word))
```

3. Output

```
Enter "help" below or click "Help" above for more information.
>>>
=============== RESTART: C:/Users/janap/OneDrive/Documents/nl1.py ==============
cars -> ('car', 'Plural Noun')
boxes -> ('box', 'Plural Noun')
cities -> ('city', 'Plural Noun')
children -> ('child', 'Plural Noun')
>>>
```

4. Use of WordNet

WordNet is a lexical database that contains English words and their relationships. It can help identify valid lemmas and irregular forms.

For example, the following code uses NLTK's WordNet lemmatizer:

```
import nltk
nltk.download("wordnet")
from nltk.stem import WordNetLemmatizer
```

```
lemmatizer = WordNetLemmatizer()

words = ["cars", "boxes", "cities", "children"]

for word in words:
    print(word, "->", lemmatizer.lemmatize(word, pos="n"))
```

Expected output:

```
cars -> car

boxes -> box

cities -> city

children -> child
```

5. Limitations of Rule-Based Finite-State Parsing

- It requires manually defined rules.
- Irregular words are difficult to handle without a lexicon.
- Some words have multiple meanings and grammatical roles.
- Incorrect rules can produce invalid roots.
- New words and social media spellings may not be recognized.

Conclusion: A combination of finite-state rules and a lexical database is more reliable than suffix removal alone.

Question 7 – Error Analysis of Morphological Feature Extraction

1. Identify the Preprocessing Error

The original program applies Porter stemming after CountVectorizer has extracted the vocabulary.

```
X = vectorizer.fit_transform(documents)

features = [

    stemmer.stem(word)

    for word in vectorizer.get_feature_names_out()

]
```

This is incorrect because the vectorizer has already created separate columns for different word forms. Stemming the feature names afterward does not combine those columns in the feature matrix.

For example, running, runners, and runs may appear as separate features even when their stems are related.

2. Corrected Program

```
from nltk.stem import PorterStemmer

from sklearn.feature_extraction.text import CountVectorizer

import re

import time

stemmer = PorterStemmer()

documents = [

    "running runners runs",

    "studies studied studying",

    "organization organized organizer"

]
```

```
def preprocess(text):
    words = re.findall(r"\b[a-zA-Z]+\b", text.lower())
   return " ".join(stemmer.stem(word) for word in words)
start = time.time()
processed_documents = [preprocess(doc) for doc in documents]
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(processed_documents)
end = time.time()
print("Processed Documents:")
for doc in processed_documents:
    print(doc)
print("\nVocabulary:")
print(vectorizer.get_feature_names_out())
print("\nVocabulary Size:", len(vectorizer.get_feature_names_out()))
print("Processing Time:", round(end - start, 6), "seconds")
```

3. Expected Output

```
Processed Documents:
run runner run
studi studi studi
organ organ organ

Vocabulary:
['organ' 'run' 'runner' 'studi']
```

Vocabulary Size: 4

Processing Time: Depends on the system

4. Vocabulary Comparison

Word group	Before stemming	After stemming
running, runners, runs	Three features	run, runner
studies, studied, studying	Three features	studi
organization, organized, organizer	Three features	organ

For the nine tokens in this small example, the vocabulary size decreases from 9 unique words to 4 stems.

5. Classification Accuracy and Processing Time

To evaluate the effect on classification, use a labeled dataset such as 20 Newsgroups and train the same classifier under two conditions:

- Before correction: Extract features without stemming.
- After correction: Stem the documents before feature extraction.

Compare vocabulary size, classification accuracy, and total preprocessing-plus-training time. Actual accuracy and timing values require running the experiment on the dataset; they cannot be established from the sample documents alone.

6. Interpretation

Stemming before feature extraction allows related word forms to share features, which can reduce vocabulary size and improve computational efficiency. However, stemming can also merge words with different meanings. Therefore, stemming does not guarantee higher classification accuracy. Lemmatization or retaining the original words may work better when preserving semantic information is important.

Conclusion: Text normalization should occur before feature extraction. This ensures that the vocabulary and feature matrix use the same normalized forms, reducing redundant features and making the preprocessing pipeline consistent.